Firmware de Telemetría ESP32 - Proyecto LaGuardia

Este repositorio contiene el código de firmware (C++/Arduino) diseñado para el microcontrolador ESP32, el cual actúa como el núcleo de captura de datos del sistema de telemetría embarcado para la maqueta del proyecto LaGuardia (FIWARE Space 2026). Su función exclusiva es la lectura en tiempo real de los sensores del avión y su transmisión inalámbrica hacia el servidor.

Descripción del Firmware

El código está diseñado para operar de forma autónoma, sin bloqueo de subprocesos y con alta tolerancia a fallos de red. Se encarga de recolectar los datos físicos de orientación (aceleración y giroscopio) y altitud, y publicarlos mediante el protocolo ligero MQTT. Además, implementa un aprovisionamiento de red dinámico, eliminando la necesidad de reescribir y compilar el código al cambiar de router o ubicación.

Hardware Configurado

- **ESP32:** Microcontrolador principal.
- **Módulo MPU6050:** Sensor inercial de 6 grados de libertad (I2C).
- **Módulo HC-SR04:** Sensor ultrasónico para telemetría de distancia (Pines Digitales).

Librerías Utilizadas

El funcionamiento del sistema se apoya en las siguientes dependencias de software, que deben estar instaladas en el entorno de desarrollo:

- **`WiFiManager.h` (junto a `DNSServer.h` y `WebServer.h`):** Gestiona la conexión de red de forma inteligente. Si el ESP32 no encuentra la red guardada en su memoria, despliega automáticamente un punto de acceso y un portal cautivo (con el SSID `Maqueta_Guardia_[MAC]`). Esto permite introducir las credenciales de la red local desde un teléfono móvil.
- **`PubSubClient.h`:** Responsable de la comunicación de telemetría IoT. Mantiene la conexión viva con el broker MQTT y se encarga de publicar los datos crudos de aceleración, rotación y distancia en los distintos *topics* designados (`la_guardia/#`).
- **`Adafruit_MPU6050.h` y `Adafruit_Sensor.h`:** Proporcionan una abstracción de alto nivel para inicializar y leer el chip MPU6050. En este código, se utilizan para establecer parámetros aeronáuticos específicos: rango de acelerómetro (8G), rango

de giroscopio (500°/s) y un filtro paso bajo (ancho de banda de 21 Hz) que suaviza las vibraciones mecánicas antes de enviar los datos.

- **`Wire.h` y `WiFi.h`:** Librerías base del núcleo del ESP32 para manejar, respectivamente, el bus de comunicación I2C del sensor y el hardware de radiofrecuencia.

Flujo Lógico del Código (`loop`)

1. **Monitor de Enlace:** Verifica el estado de la conexión MQTT. Si se pierde, entra en una rutina de reconexión controlada, asegurándose primero de que el enlace WiFi principal siga activo para no bloquear el chip.
2. **Muestreo Inercial:** Solicita los eventos físicos al MPU6050, extrayendo los vectores de fuerza G (X, Y, Z) y las velocidades de rotación (X, Y, Z).
3. **Muestreo Ultrasónico:** Dispara el pin `TRIG` del HC-SR04 durante 10 microsegundos, lee el tiempo de retorno en el pin `ECHO` y aplica la constante de la velocidad del sonido para obtener la altitud en centímetros.
4. **Casting y Publicación:** Convierte los valores en coma flotante a cadenas de caracteres (`String().c_str()`) y los inyecta en la red MQTT.
5. **Regulación de Cadencia:** Aplica una pausa no obstructiva (`delay(100)`) que garantiza una tasa de refresco estable de 10 Hz. Esto mantiene la simulación fluida sin saturar la cola de mensajes del broker Mosquitto.